AKS Observability Platform - GUI Runbook

Portal and Azure DevOps click-by-click implementation guide

Azure | AKS | DevOps | Observability

GUI path: Azure Portal + Azure DevOps + GitHub validation

Document	Prepared for	Date	Repository
AKS Observability Platform - GUI Runbook	Rizwan Siddiqui	30 July 2026	RizwanSid7/azure-monitor-aks-observability-capstone

Use of this document
This PDF is written so another learner or engineer can understand why this setup exists, how to implement it, how to validate it, and how to present it in a demo.

1. Project Purpose and Problem Statement

Section goal
Understand why this setup exists before clicking through the portal.

This project demonstrates how a small application can be containerized, deployed to Azure Kubernetes Service (AKS), exposed using Application Gateway Ingress Controller, monitored with Azure Monitor and Container Insights, and delivered through Azure DevOps pipeline.

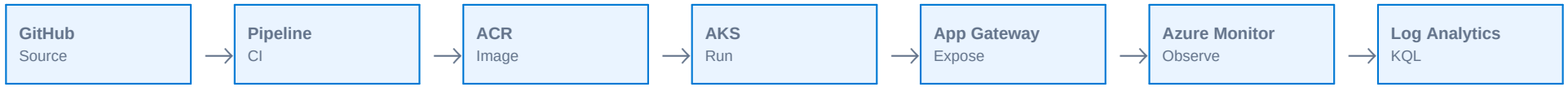
The business problem it solves is visibility. Without observability, teams can deploy applications but cannot confidently answer: Is the app reachable? Are pods healthy? Are errors happening? Did a new image reach the registry? Can the system recover from pod failure?

Problem	How this setup addresses it
Application deployment inconsistency	Docker packages app + dependencies, ACR stores trusted images, AKS runs the containers.
No safe internet exposure pattern	Application Gateway exposes traffic with layer 7 routing and can be extended with WAF/HTTPS.
No operational visibility	Container Insights and Log Analytics provide metrics, logs, and KQL-based troubleshooting.
Manual image delivery	Azure DevOps pipeline builds and pushes images automatically.
Limited recovery proof	The /crash endpoint and pod restart verification show Kubernetes self-healing behavior.

Best fit
This setup is best for cloud-native web apps, internal tools, APIs, microservices demos, DevOps portfolios, AKS observability labs, and pre-production validation environments.

2. Architecture Overview

Section goal
See the end-to-end flow before creating or verifying resources.



High-level runtime flow:

- User opens the Application Gateway public endpoint.
- Application Gateway Ingress Controller maps the Kubernetes Ingress object to Application Gateway rules.
- Traffic is routed to Kubernetes Service and then to AKS pods.
- Container Insights/AMA collects container logs, pod status, node data, and performance data.
- Log Analytics and KQL are used for investigation and reporting.

Architecture note
In this lab, the cluster used one node and Basic ACR for simplicity. For production, use separate environments, multiple node pools, private networking, HTTPS, RBAC, and governance policies.

3. Prerequisites

Section goal
Prepare access, tools, and decisions before starting.

Prerequisite	Details
Access	Azure subscription, permission to create resource groups, AKS, ACR, Log Analytics, Application Gateway, and role assignments where needed.
Local tools	Browser, Git, Docker Desktop, Azure CLI, kubectl, PowerShell, and an editor such as VS Code/Notepad.
Azure DevOps	Organization/project access, ability to create pipeline, service connection, and PAT for self-hosted agent.
Naming	Use consistent names: rg-aks-observability-capstone, acrrizwanobs268, aks-observability-capstone, law-aks-observability.
Security	Never share PAT token, ACR passwords, subscription IDs, tenant IDs, object IDs, or full resource IDs in screenshots.
Cost	Use this as a short-term lab unless budget is approved. Delete the resource group after recording and validating proof.

4. GitHub Repository - GUI Proof

Open the repository in GitHub and verify that it contains app, k8s, monitoring, screenshots, docs, Dockerfile, and pipeline YAML.

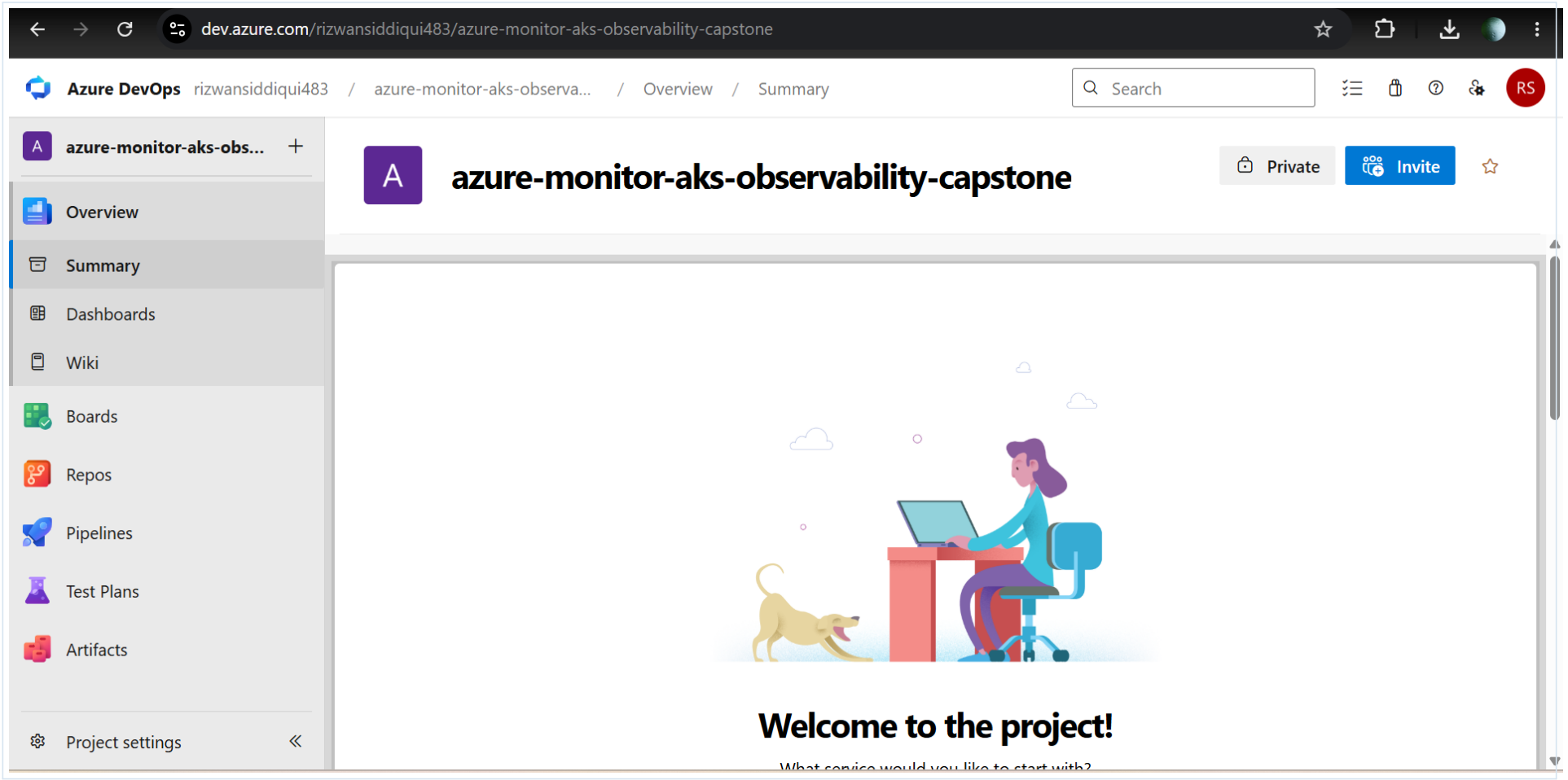
```
Admin@Rizwan MINGW64 ~/Documents/azure-monitor-aks-observability-capstone (main)
$ cd ~/Documents/azure-monitor-aks-observability-capstone

git status
git log --oneline --max-count=3
On branch main
nothing to commit, working tree clean
d6376ac (HEAD -> main) Tested observability app locally with Docker
879d383 Initial AKS observability capstone project structure

Admin@Rizwan MINGW64 ~/Documents/azure-monitor-aks-observability-capstone (main)
$ git remote add origin https://github.com/RizwanSid7/azure-monitor-aks-observability-capstone.git
git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 25, done.
Counting objects: 100% (25/25), done.
Delta compression using up to 8 threads
Compressing objects: 100% (22/22), done.
Writing objects: 100% (25/25), 489.87 KiB | 15.31 MiB/s, done.
Total 25 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/RizwanSid7/azure-monitor-aks-observability-capstone.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

Admin@Rizwan MINGW64 ~/Documents/azure-monitor-aks-observability-capstone (main)
$
```

07-github-repository-created.png

Demo explanation
This is the central source of truth for the project. The README explains the purpose and the folders show implementation artifacts.

5. Azure DevOps Project - GUI Proof

Create or open the Azure DevOps project. This project hosts the pipeline configuration and service connection.



37-azure-devops-project-created.png

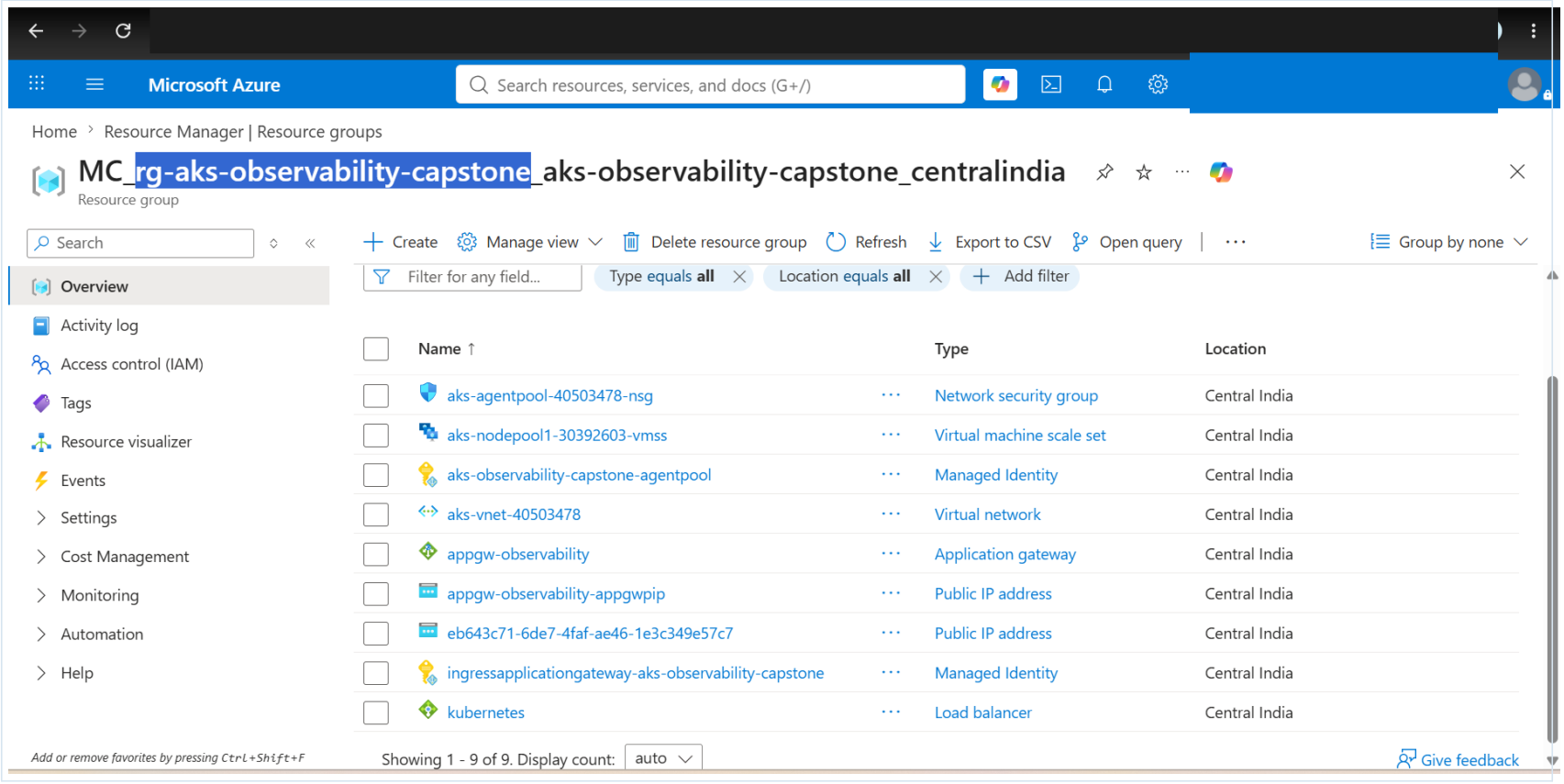
Demo explanation
Azure DevOps is used here to show real CI/CD flow, not just manual image push.

6. Azure Portal Resource Setup - GUI Steps

Section goal
Create or verify the main Azure resources through the portal.

Portal path for a resource group:

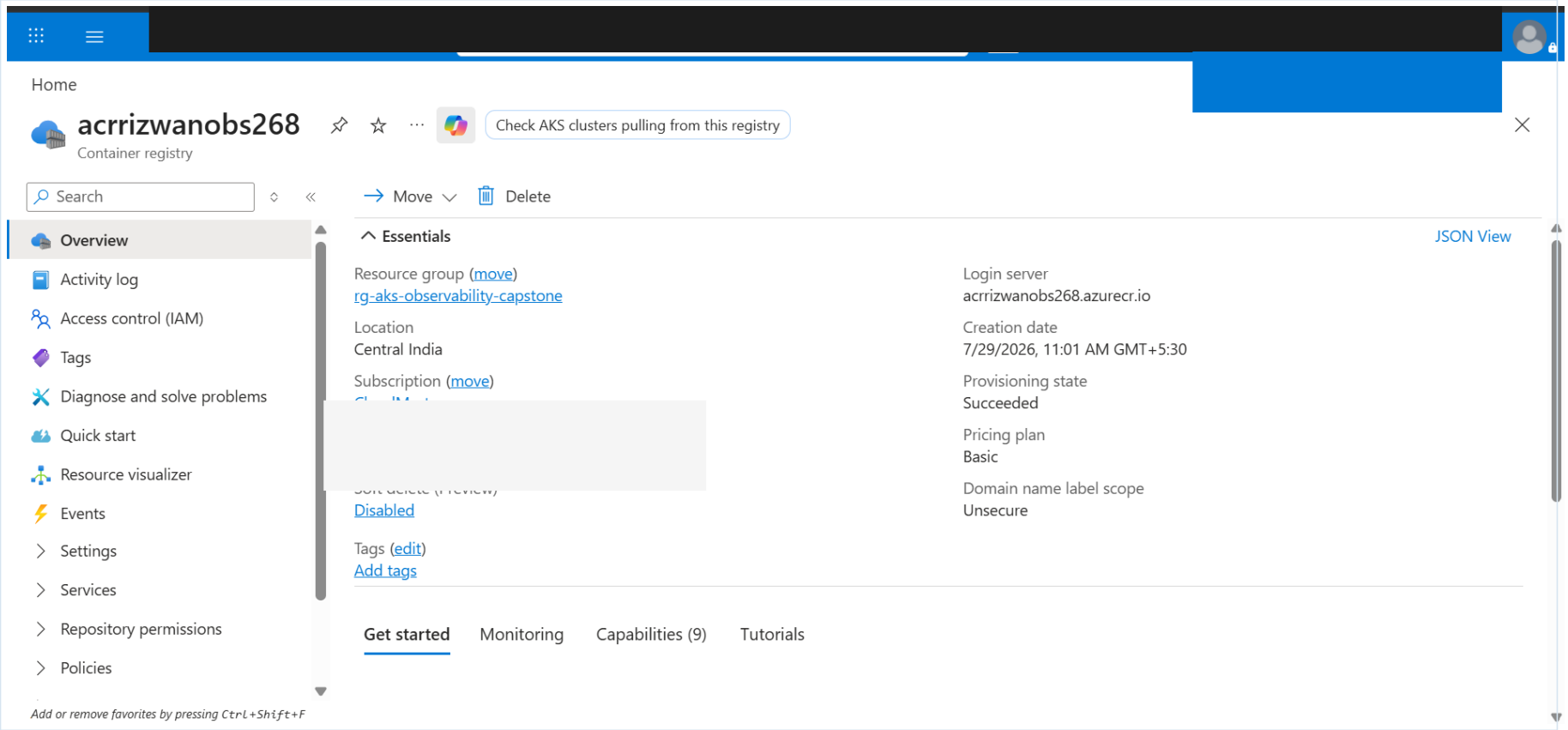
1. Go to Azure Portal.
2. Search for Resource groups.
3. Select Create.
4. Choose subscription, enter resource group name, select region Central India, and create.
5. Use this group to keep resources easy to manage, review, and delete.



Resource group overview showing AKS managed resources and Application Gateway related resources.

Portal path for ACR:

1. Search Container registries.
2. Select Create.
3. Choose the resource group, enter a globally unique registry name, select region, choose Basic for lab, and create.
4. After images are pushed, open Repositories to verify image name and tags.



ACR overview. Sensitive IDs are redacted.

7. ACR Tags After Pipeline

After the Azure DevOps pipeline succeeds, check that the repository contains both latest and Build ID tags.

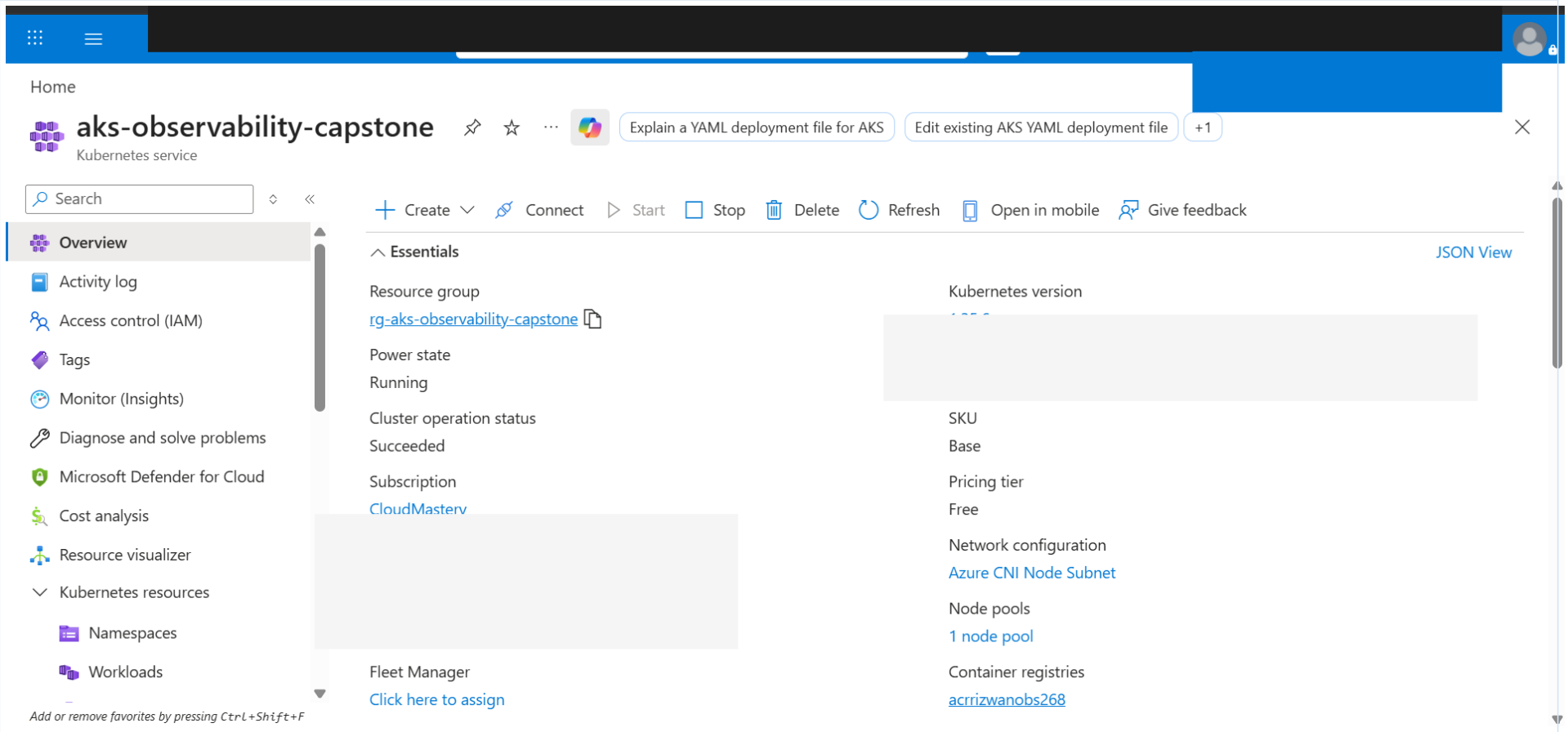
```
}
PS C:\WINDOWS\system32> az acr repository show-tags --name acrrizwanobs268 --repository observability-app --output table
Result
--------
13
latest
PS C:\WINDOWS\system32>
```

42-acr-image-tags-after-pipeline.png

Demo explanation
This proves that the pipeline pushed the image to ACR successfully.

8. AKS Overview - Portal Validation

Open AKS > Overview to validate cluster status, Kubernetes version, node pool, network configuration, and linked ACR.



46-aks-overview.png

Demo explanation
The AKS overview confirms the cluster is running and connected to the container registry.

9. Creating AKS with Monitoring and AGIC - GUI Method

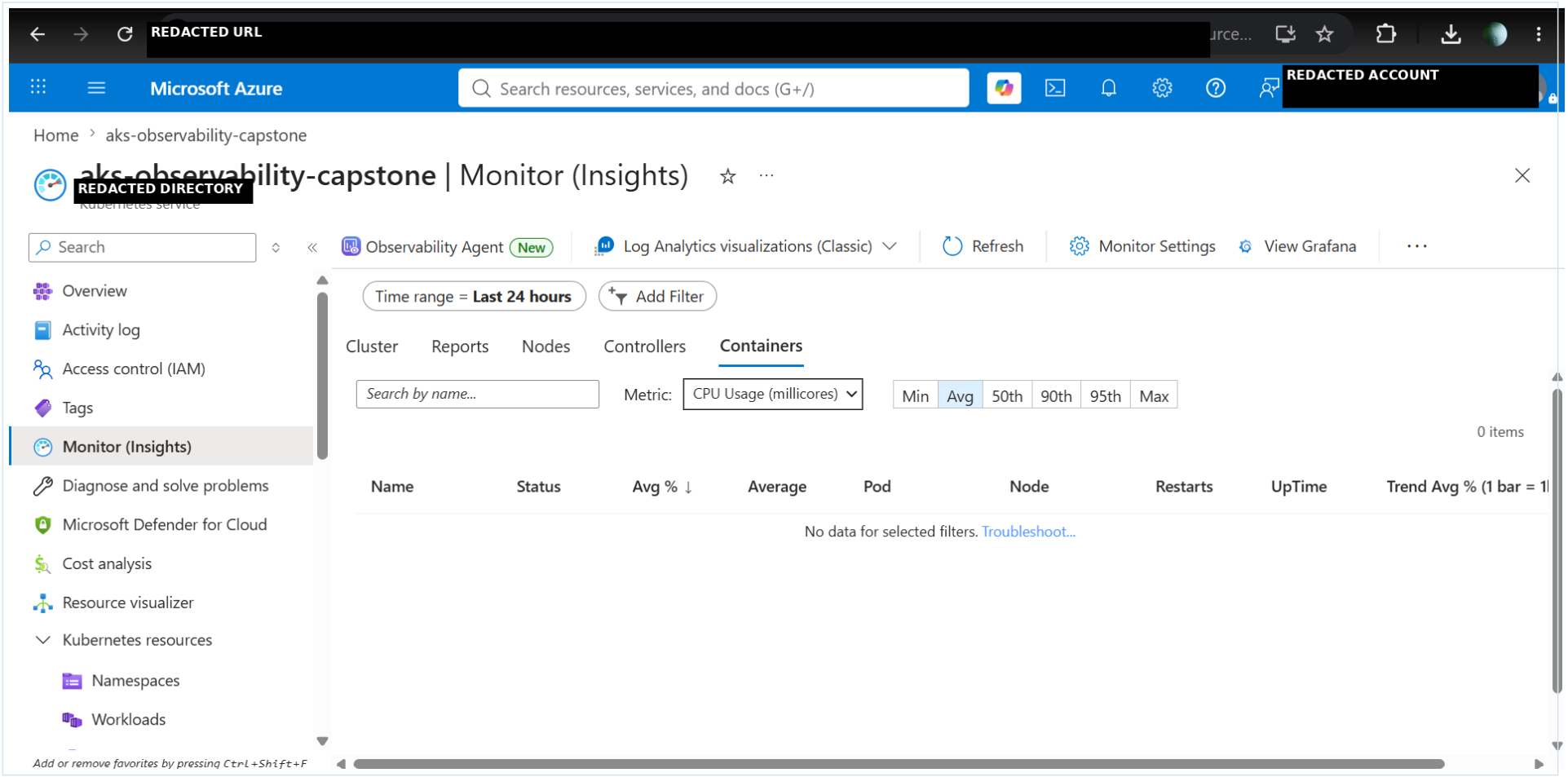
Section goal
These are the portal-equivalent steps for the cluster used in the project.

1. Search for Kubernetes services and select Create.
2. Choose subscription and resource group.
3. Enter cluster name aks-observability-capstone and region Central India.
4. Use a lab-friendly node pool size for testing. For production, use multiple nodes and zones.
5. On integrations/monitoring, enable Container Insights and connect the Log Analytics Workspace.
6. Enable Application Gateway Ingress Controller or create the Application Gateway add-on during cluster creation.
7. Review and create. Wait until provisioning state is Succeeded.

Production note
For production workloads, prefer private cluster design, multiple availability zones where supported, autoscaling, network policies, workload identity, image scanning, and HTTPS/WAF at ingress.

10. Container Insights - Portal Validation

Open AKS > Monitor (Insights) to verify that Container Insights is enabled and visible.

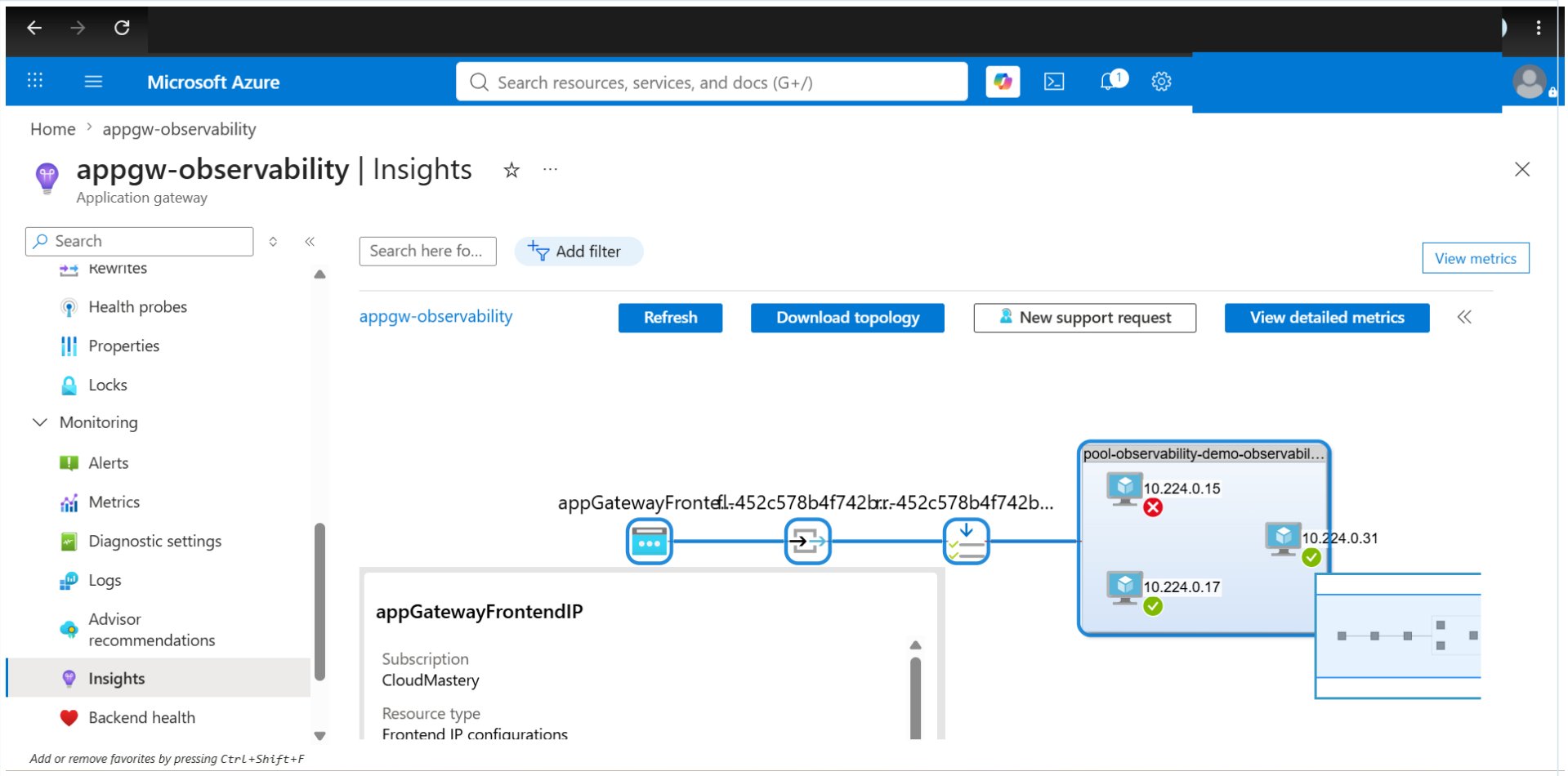


32-container-insights-overview.png

Demo explanation
Container Insights is used for pod, node, controller, and container monitoring.

11. Container Insights Containers/Pods View

Use the Containers or Pods view to check CPU/memory metrics and pod/container health. In this lab the table may show no data immediately because ingestion can take time or filters may not match.



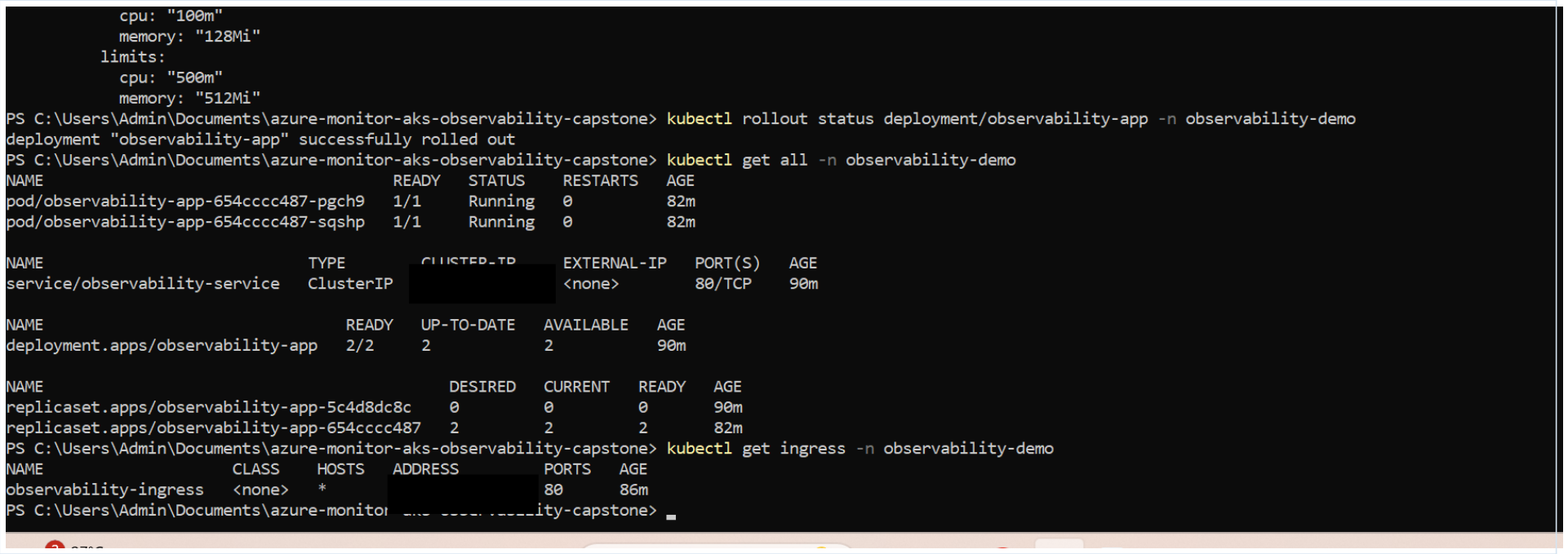
50-container-insights-containers-pods.png

Demo explanation
Even when portal data is delayed, we also validate using kubectl logs and add-on status.

12. Kubernetes Workload Validation in Portal

Section goal
Use AKS Kubernetes resources menu for visual proof.

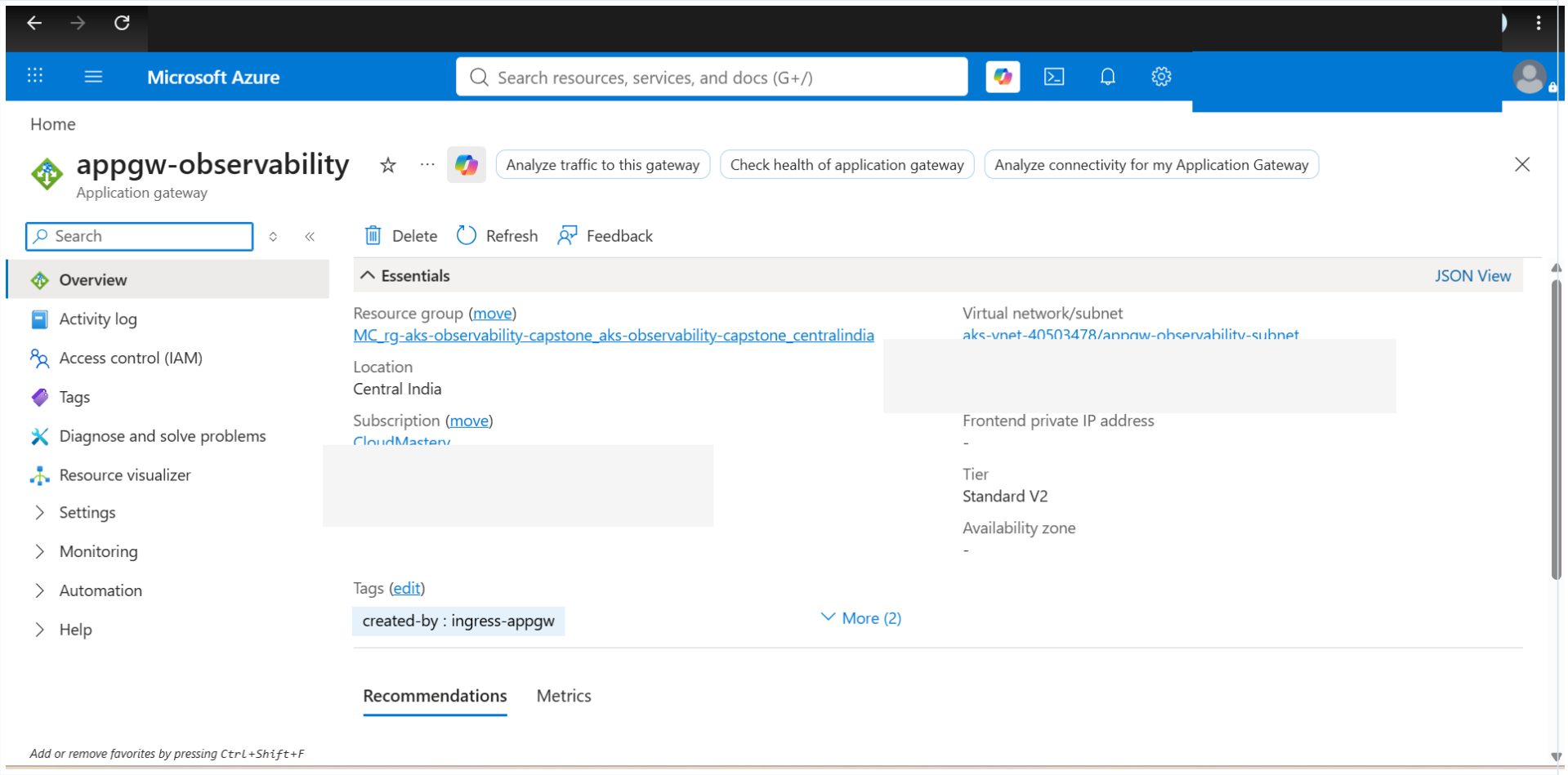
1. Open AKS resource.
2. Go to Kubernetes resources > Namespaces to verify observability-demo.
3. Go to Workloads or Pods to verify observability-app deployment and pods.
4. Use Services and ingresses to validate service and ingress objects.



CLI proof of Kubernetes objects running, useful when the portal view is loading or delayed.

13. Application Gateway Overview

Open appgw-observability > Overview to verify frontend public IP, subnet, SKU, and status.

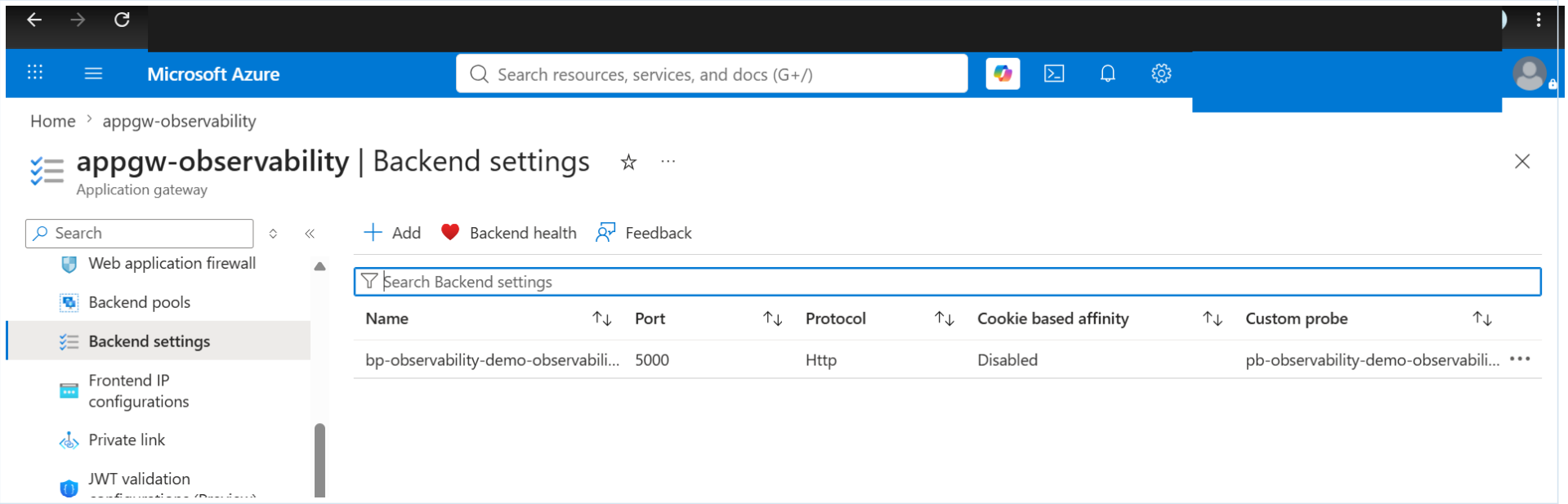


48-application-gateway-overview.png

Demo explanation
Application Gateway is the layer 7 entry point for external traffic to the AKS application.

14. Application Gateway Backend Health / Insights

Open Application Gateway > Backend health or Insights to verify backend pool health and topology.



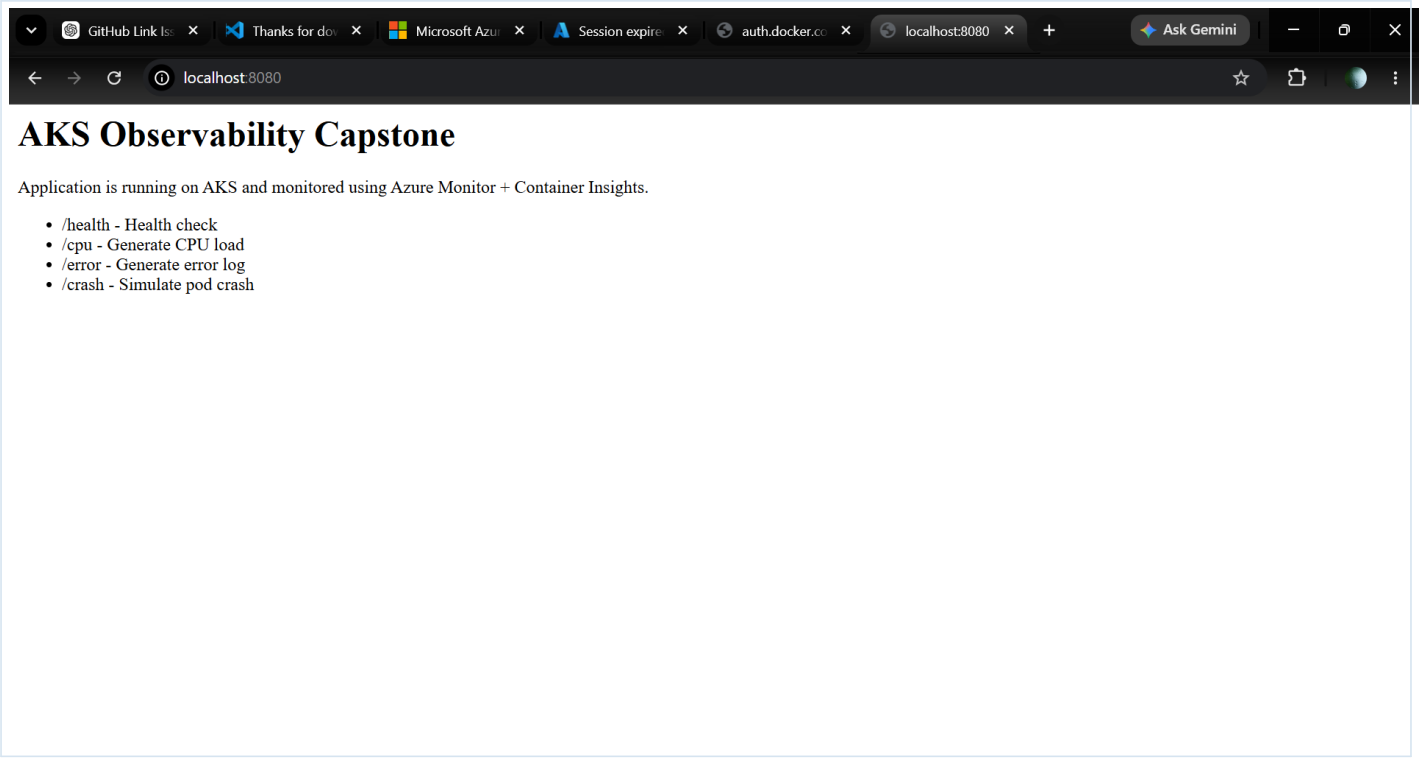
49-application-gateway-backend-health.png

Demo explanation
This helps troubleshoot whether Application Gateway can reach the AKS pod backends.

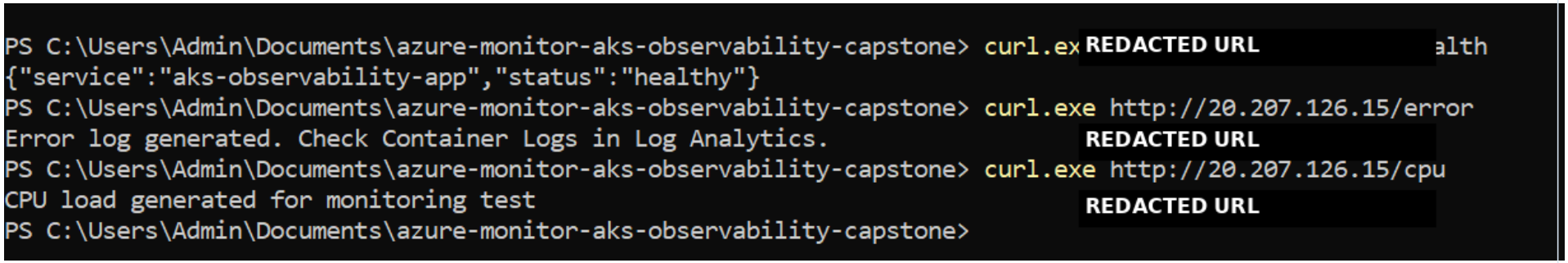
15. Application Endpoint Testing - GUI

Section goal
Test the endpoint from browser and use URL paths for observability scenarios.

URL	Expected result	Purpose
http://<app-gateway-ip>/	Home page	Confirms app is reachable.
/health	JSON status healthy	Health check endpoint.
/error	Error log generated	Generates application log for KQL/log testing.
/cpu	CPU load generated	Creates CPU load for monitoring test.
/crash	Connection may close/pod restarts	Demonstrates Kubernetes recovery behavior.



Local browser proof used before cloud deployment.



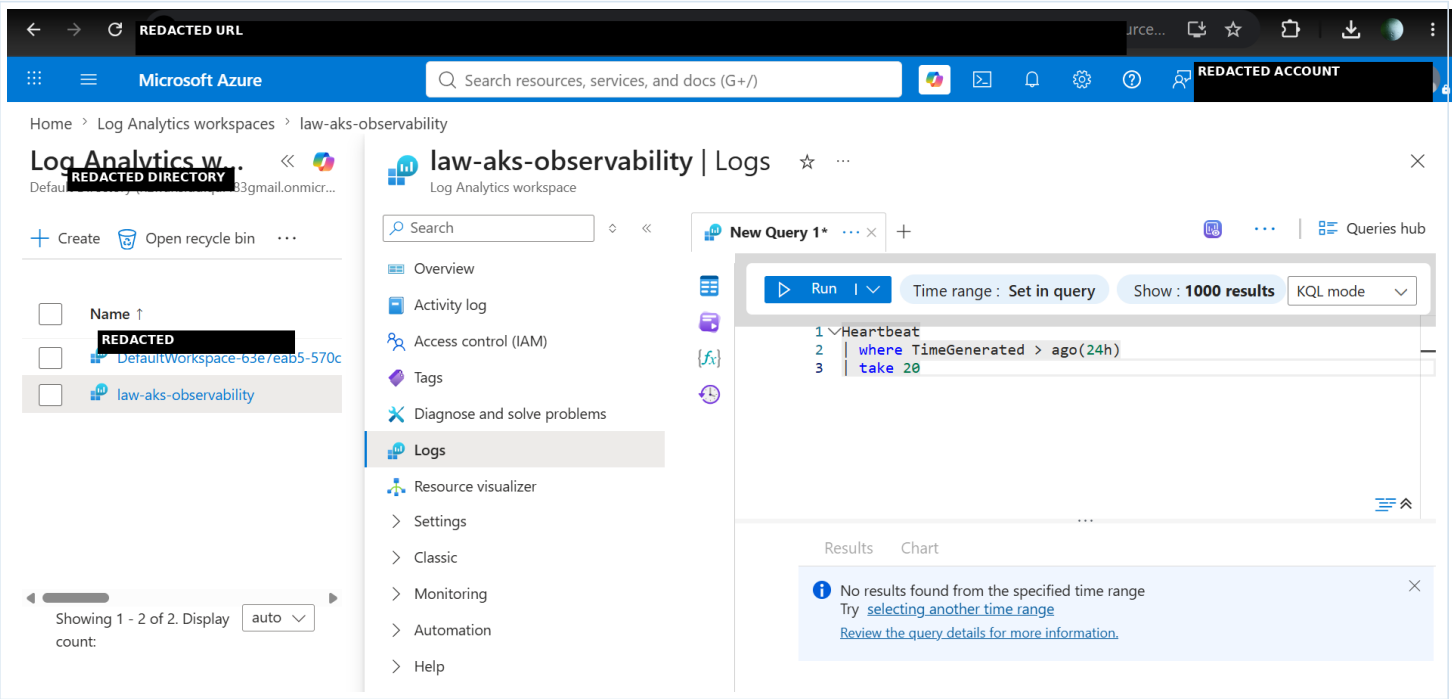
Application Gateway endpoint test using app paths.

16. Log Analytics and KQL in Portal

Section goal
Use the Logs blade to query collected data.

1. Open Log Analytics workspace law-aks-observability.
2. Select Logs.
3. Use KQL mode.
4. Run table discovery query or targeted queries for ContainerLogV2, KubePodInventory, Heartbeat, InsightsMetrics, or Perf.
5. If no data appears, verify add-on status, AMA pods, time range, namespace filters, and ingestion delay.

```
search *
| where TimeGenerated &gt; ago(24h)
| summarize Count=count() by $table
| order by Count desc

ContainerLogV2
| where TimeGenerated &gt; ago(24h)
| where PodNamespace == &quot;observability-demo&quot;
| project TimeGenerated, PodName, ContainerName, LogMessage
| order by TimeGenerated desc
```



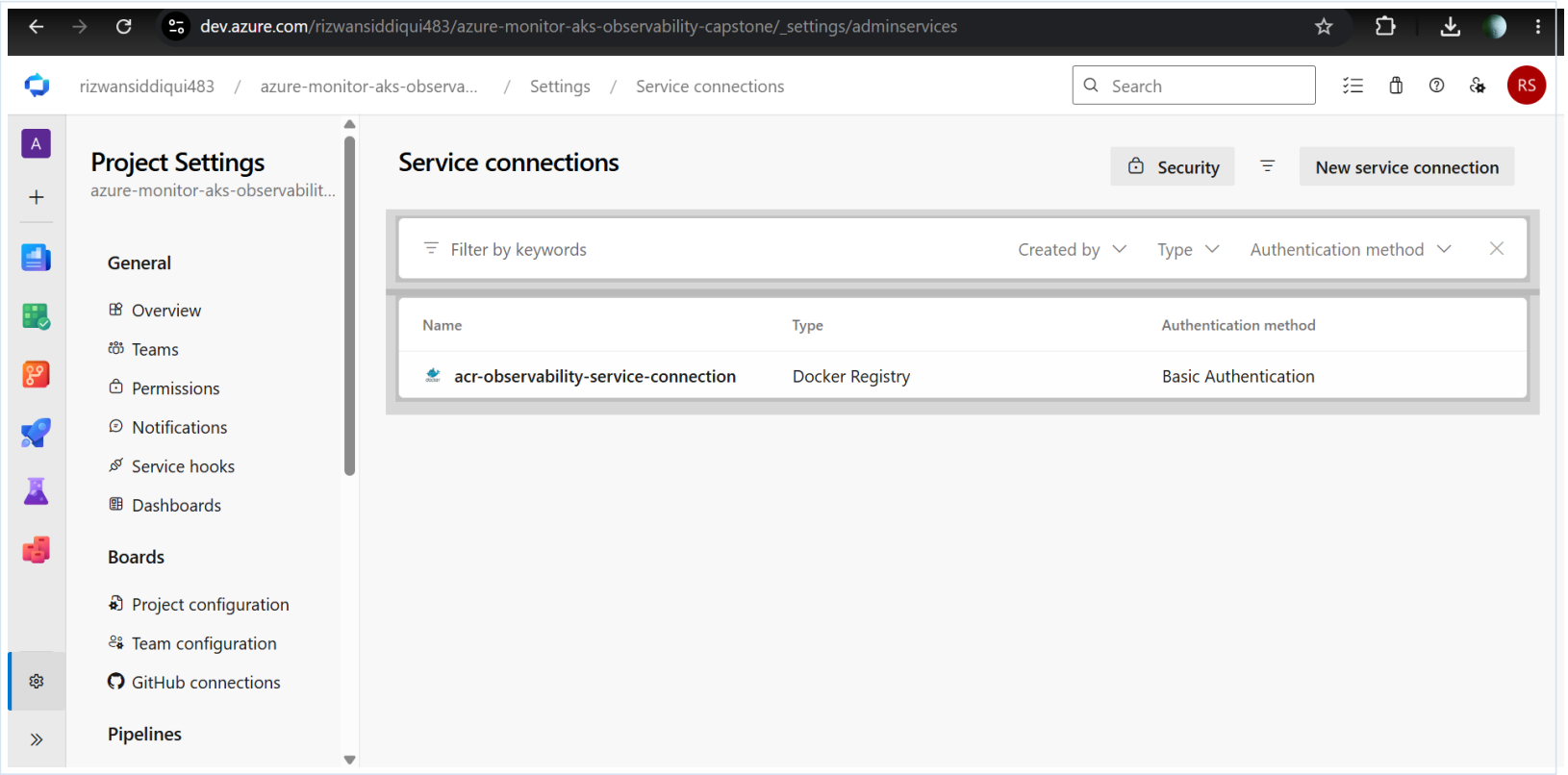
Log Analytics query screen. No data scenarios are documented as part of troubleshooting.

17. Azure DevOps GUI - Service Connection and Pipeline

Section goal
Create the pipeline connection and validate success.

Service connection path:

1. Azure DevOps project > Project settings.
2. Select Service connections.
3. Create New service connection.
4. Choose Docker Registry.
5. Use ACR login server and registry credentials.
6. Name it acr-observability-service-connection.
7. Grant access permission to all pipelines, verify, and save.



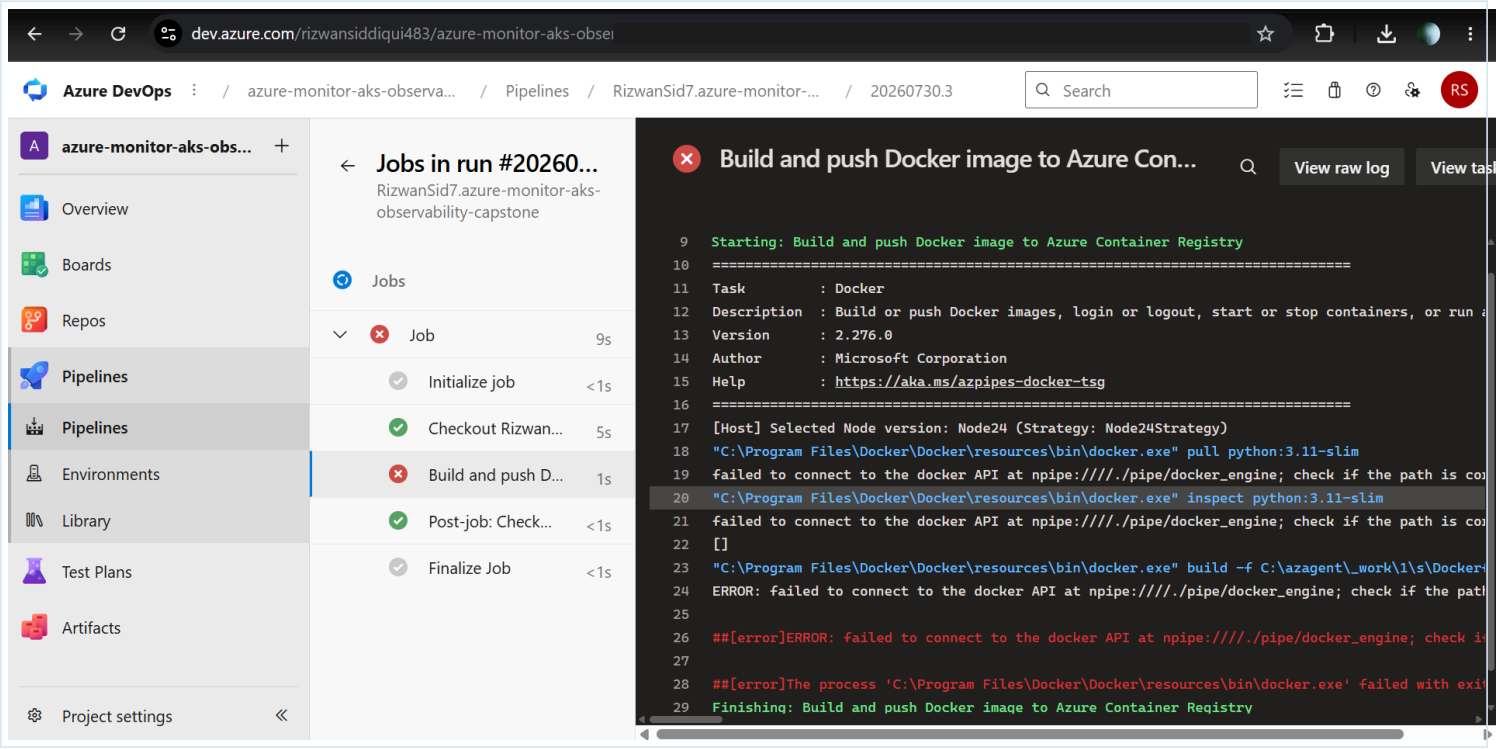
Docker Registry service connection proof.

Security note
Use short-lived credentials for lab validation. In production, prefer managed identities/service principals with least privilege and avoid keeping ACR admin enabled.

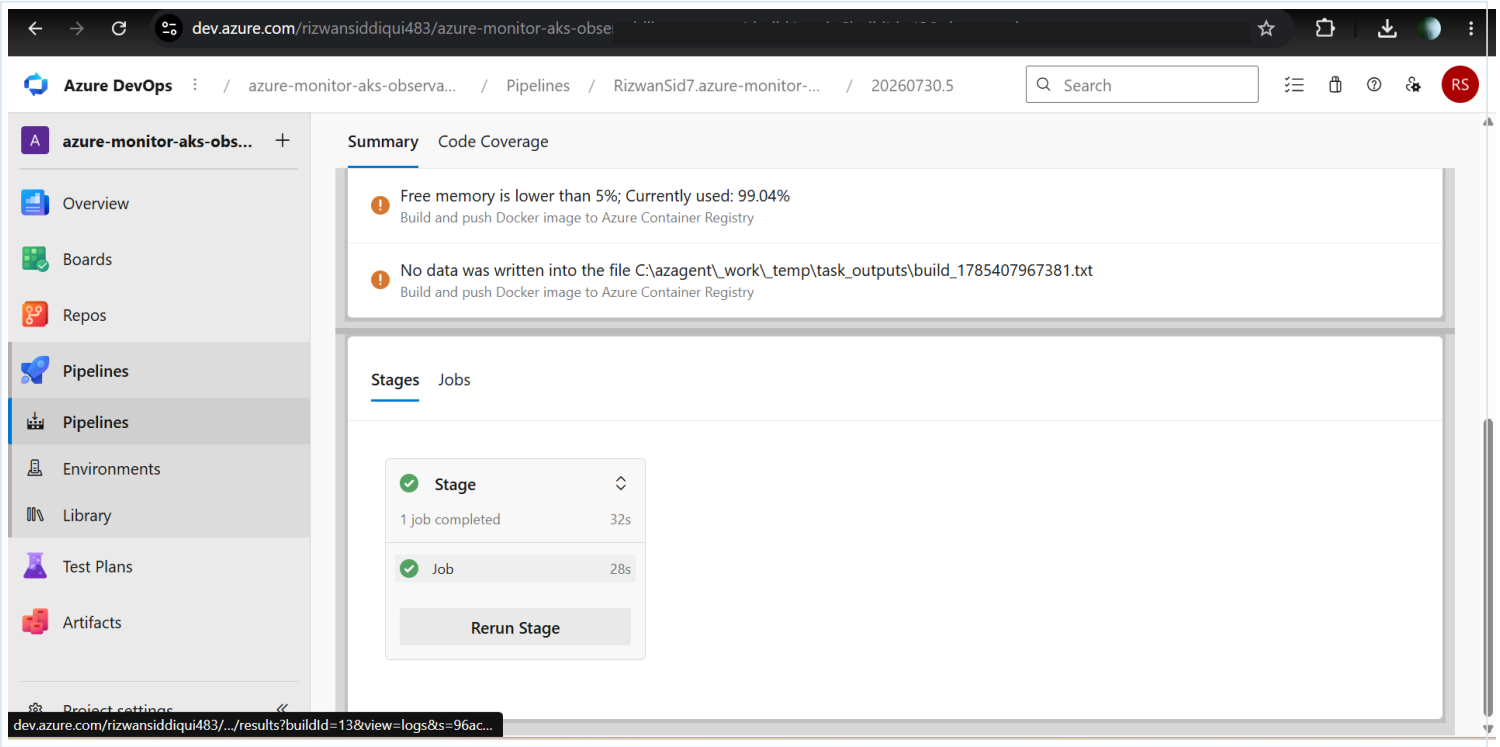
Pipeline path:

1. Azure DevOps > Pipelines > Create Pipeline.
2. Select GitHub and authorize repository access.
3. Choose existing Azure Pipelines YAML file.
4. Select azure-pipelines.yml from main branch.
5. Run the pipeline.
6. If it queues, start the self-hosted agent.
7. If Docker daemon fails, start Docker Desktop and rerun.

8 If ACR authentication fails, update the service connection credentials and rerun.



Troubleshooting proof: pipeline failed when Docker daemon was not running.



Successful Azure DevOps pipeline run.

18. Demo Video Flow

Section goal
Record before deleting resources.

1. Start with GitHub repository and README.
2. Show architecture/resource visualizer or resource group overview.
3. Show ACR image tags.
4. Show AKS overview and pods.
5. Show Application Gateway overview and backend health.
6. Open application URL and test /health, /error, /cpu.
7. Show kubectl logs and pod restart proof.
8. Show Azure DevOps pipeline success and ACR tag created by pipeline.
9. Close with what problem it solves and what you would improve for production.

Do not show
PAT tokens, ACR passwords, full subscription ID, tenant/client/object IDs, workspace customer ID, private IPs, and account email.

19. Cost Effectiveness and Cleanup

Section goal
Keep it cost safe.

This lab is useful and cost-effective for short demos if resources are deleted quickly. It is not cost-effective to leave running without a reason because AKS nodes, Application Gateway, ACR storage, public IPs, and Log Analytics ingestion/retention may keep generating charges.

Component	Cost note
AKS	Control plane may be free for standard tier choices, but VM nodes are billed while allocated.
Application Gateway	Billed while provisioned/available and for processed traffic/capacity depending on SKU.
Azure Monitor / Log Analytics	Billed mainly by data ingestion and retention beyond included periods/settings.
ACR	Billed by registry tier/storage/operations according to tier.
Public IP and managed resources	Small but real charges may apply. Always delete the whole resource group for labs.

```
# Cleanup after demo
az group delete --name rg-aks-observability-capstone --yes --no-wait

# Verify deletion
az group exists --name rg-aks-observability-capstone
# Expected output: false
```

Production Readiness and Best Practices

Area	Recommendation
Networking	Use private AKS where possible, private endpoints for ACR, restricted inbound traffic, and WAF when exposing internet workloads.
Identity	Use managed identity, least-privilege RBAC, separate service connections, and avoid long-lived admin passwords.
Delivery	Use protected branches, pull requests, pipeline approvals, environment gates, and image scanning before deployment.
Reliability	Use multiple node pools, autoscaler, readiness/liveness probes, rolling updates, PodDisruptionBudgets, and zones where available.
Observability	Use Container Insights, Prometheus/Grafana where needed, KQL queries, alert rules, dashboards, and clear incident runbooks.
Cost	Use budgets and alerts, right-size nodes, set log retention, stop/delete lab resources, and avoid leaving Application Gateway and AKS nodes running unnecessarily.
Security	Use HTTPS/TLS, Key Vault, Defender for Cloud, network policies, secret management, and restrict public IP exposure.

Official Microsoft Documentation References

Use these primary Microsoft sources whenever you need to verify steps, product behavior, limits, and pricing. Pricing pages are estimates and should be checked again before production decisions.

Topic	Official link
AKS monitoring and Container Insights	https://learn.microsoft.com/en-us/azure/aks/monitor-aks
Application Gateway Ingress Controller overview	https://learn.microsoft.com/en-us/azure/application-gateway/ingress-controller-overview
AGIC annotations	https://learn.microsoft.com/en-us/azure/application-gateway/ingress-controller-annotations
Create AGIC add-on for a new AKS cluster	https://learn.microsoft.com/en-us/azure/application-gateway/tutorial-ingress-controller-add-on-new
Azure Container Registry documentation	https://learn.microsoft.com/en-us/azure/container-registry/
ACR quickstart with Azure CLI	https://learn.microsoft.com/en-us/azure/container-registry/container-registry-get-started-azure-cli
ACR Tasks: build and run container images	https://learn.microsoft.com/en-us/azure/container-registry/container-registry-quickstart-task-cli
Azure Monitor Log Analytics overview	https://learn.microsoft.com/en-us/azure/azure-monitor/logs/log-analytics-overview
Log queries in Azure Monitor	https://learn.microsoft.com/en-us/azure/azure-monitor/logs/log-query-overview
Azure CLI Log Analytics query	https://learn.microsoft.com/en-us/azure/azure-monitor/logs/api/request-format
Azure DevOps self-hosted agents	https://learn.microsoft.com/en-us/azure/devops/pipelines/agents/docker?view=azure-devops
Docker task for Azure Pipelines	https://learn.microsoft.com/en-us/azure/devops/pipelines/tasks/reference/docker-v0?view=azure-pipelines
AKS best practices	https://learn.microsoft.com/en-us/azure/aks/best-practices
AKS baseline architecture	https://learn.microsoft.com/en-gb/azure/architecture/reference-architectures/containers/aks/baseline-aks
AKS pricing	https://azure.microsoft.com/en-us/pricing/details/kubernetes-service/
Application Gateway pricing	https://azure.microsoft.com/en-in/pricing/details/application-gateway/
Azure Monitor pricing	https://azure.microsoft.com/en-in/pricing/details/monitor/
Azure Container Registry pricing	https://azure.microsoft.com/en-gb/pricing/details/container-registry/